

ANT 系列分组密码算法*

陈师尧^{1,2}, 樊燕红^{1,2}, 付 勇^{1,2}, 黄鲁宁^{1,2}, 王美琴^{1,2}

1. 山东大学 网络空间安全学院, 青岛 266237
2. 密码技术和信息安全教育部重点实验室, 青岛 266237

通信作者: 王美琴, E-mail: mqwang@sdu.edu.cn

摘 要: 我们设计了一款新的系列分组密码算法—ANT, 其包含 3 个版本, 根据分组长度/密钥长度可以分别记为 ANT-128/128、ANT-128/256 和 ANT-256/256. ANT 算法采用了经典的 Feistel 结构, 轮函数采用比特级的设计, 仅包含与操作, 循环移位操作和异或操作 (AND-Rotation-XOR). 结合 Expand-then-compress 的设计思想, 使 ANT 算法达到了较好的扩散速度. 得益于精心构造的比特级轮函数, 在保证算法达到较高安全性的同时, 还具有出色的硬件性能, 非常适合轻量级实现. 在硬件实现环境 (HJTC 110 nm 标准元件库) 下, ANT-128/128 加解密基于轮实现的硬件面积仅为 3220 GE. 而轮函数仅采用与操作作为非线性操作, 相比传统的 S 盒算法, ANT 算法在侧信道防护实现上更具优势. 在软件方面, ANT 算法在设计过程中就充分考虑到 bitslice 的实现速率. 测试结果也表明, ANT 算法具有出色的多路软件性能. 针对现有常见的攻击方法, 我们对 ANT 算法进行了全面的安全性分析, 分析结果表明 ANT 系列算法各个版本均具有较高的安全冗余.

关键词: 轻量级分组密码; Feistel 结构; AND-Rotation-XOR 操作; 吞面比; bitslice 实现

中图分类号: TP309.7 **文献标识码:** A DOI: 10.13868/j.cnki.jcr.000338

中文引用格式: 陈师尧, 樊燕红, 付勇, 黄鲁宁, 王美琴. ANT 系列分组密码算法[J]. 密码学报, 2019, 6(6): 748–759.

英文引用格式: CHEN S Y, FAN Y H, FU Y, HUANG L N, WANG M Q. On the design of ANT family block ciphers[J]. Journal of Cryptologic Research, 2019, 6(6): 748–759.

On the Design of ANT Family Block Ciphers

CHEN Shi-Yao^{1,2}, FAN Yan-Hong^{1,2}, FU Yong^{1,2}, HUANG Lu-Ning^{1,2}, WANG Mei-Qin^{1,2}

1. School of Cyber Science and Technology, Shandong University, Qingdao 266237, China
2. Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education, Shandong University, Qingdao 266237, China

Corresponding author: WANG Mei-Qin, E-mail: mqwang@sdu.edu.cn

* 基金项目: 国家自然科学基金 (61572293, 61502276, 61692276); 国家密码发展基金 (MMJJ20170102); 山东省重大科技创新工程 (2017CXGC0704); 山东省自然科学基金 (ZR2016FM22)

Foundation: National Natural Science Foundation of China (61572293, 61502276, 61692276); National Cryptography Development Fund (MMJJ20170102); Major Scientific and Technological Innovation Projects of Shandong Province, China (2017CXGC0704); Natural Science Foundation of Shandong Province, China (ZR2016FM22)

收稿日期: 2019-11-20 定稿日期: 2019-12-10

Abstract: This paper presents a new family of block ciphers named as ANT. According to block-size/keysize, it has three versions, namely ANT-128/128, ANT-128/256, and ANT-256/256. The ANT cipher adopts the classical Feistel structure and has a bit-based round function, which is composed of AND, Rotation, and XOR operations. Combined with the Express-then-compress design principle, ANT achieves fast speed of diffusion. The bit-based round function ensures ANT a high level security and a competitive performance in hardware implementation, and is very suitable for lightweight implementation. Since AND operation is the only non-linear operation, ANT also has an obvious advantage for the protective implementations against side-channel attack. Considering software performance, the bitslice efficiency was taken into consideration during the algorithm design, making ANT to have an efficient bitslice implementation. Some state-of-the-art cryptanalytic methods have been conducted on ANT, which show that all versions of ANT have high security margins.

Key words: lightweight block cipher; Feistel construction; AND-Rotation-XOR operations; throughput/area ratio; bitslice implementations

1 引言

1.1 背景

近年来,随着日常生活中物联网设备越来越多,这些资源受限设备下的数据传输安全也越来越受到人们的重视.传统的分组密码如 AES^[1] 等并不适合一些资源受限的环境.因此,轻量级的分组密码成为了当前的一个研究热点.密码学界也设计了许多轻量级的分组密码,如 LED^[2]、LBlock^[3]、Rectangle^[4]、PRINCE^[5] 和 Zorro^[6] 等.其中,一些高效的轻量级算法已经成为了 ISO/IEC 标准,如 PRESENT^[7] 和 CLEFIA^[8].伴随着 5G 技术的广泛应用,物联网所承载的通讯设备的数量必然会与日俱增.因此,为了更好地适应物联网设备多样化以及轻量化的趋势, NIST^[9] 在征集新的标准密码算法时也自然会考虑到这些应用场景和需求.

然而,学界并未对轻量级给出严格的定义,不同的受限应用环境也有不同的需求.比如,面积小(尺寸受限的芯片设备)、功率低(被动式 RFID)、能耗少(电池供能的设备)以及延时低(磁盘加密).因此,人们通常关注的指标有功率、能耗、硬件面积和吞吐量.但当把这些指标都纳入考虑范围,不同轻量级算法之间的比较自然是一项非常复杂的工作.通常来说,硬件面积是大家公认的一个主要性能指标.近年来,人们也提出了一个更全面的指标—吞面比,来衡量一个密码算法硬件实现过程中性能与资源消耗折衷的能力.在一定程度上,一个密码算法功率和能耗的高低也可以通过其吞面比反映出来.正是在这些指标需求的推动下,近年来涌现出许多优秀的轻量级密码算法,其中大部分都是 SPN 结构的算法,如 SKINNY^[10] 和 GIFT^[11] 等.与之相比,Feistel 结构的算法由于扩散性较慢,并未受到轻量级设计的青睐.但 Feistel 结构具有天然的加解密一致的优势,这对硬件实现非常有利.因此,如何达到更快的扩散速度,对基于 Feistel 结构的轻量级分组密码设计至关重要,也决定着其是否能发挥出优异的硬件性能.

结合这样的背景和需求,我们致力于设计一款适合轻量级实现,同时具有较好软件性能且安全强度高的分组密码算法.

1.2 主要贡献

本文给出了一个主要面向硬件实现兼具出色多路软件性能的分组密码算法—ANT,其包含三个版本: ANT-128/128、ANT-128/256 和 ANT-256/256.其主要特点如下:

- **实现效率高.** ANT 算法采用经典的 Feistel 结构,具有加解密一致的优势.而精心构造的基于比特级的轮函数,仅采用了与操作、循环移位操作和异或操作 (AND-Rotation-XOR).这些设计理念的结合,使 ANT 算法的硬件实现性能优异,所有版本均具有较小的面积和较高的吞面比.其中, ANT-128/128 在硬件实现平台 (HJTC 110 nm 标准元件库) 下,加解密基于轮的实现仅需硬件面积 3220 GE.软件方面,在设计过程中充分考虑了 bitslice 的实现性能,因此 ANT 算法具有出色的多路软件实现速率.

- **安全强度高.** 我们基于可满足性问题 (Boolean satisfiability problem, SAT) 的自动化搜索方法^[12]对现有常见的攻击方法进行了分析. 分析结果表明 ANT 算法能够抵抗现有攻击方法, 各版本具有充分的安全冗余.
- **侧信道防护代价小.** 目前的侧信道攻击大多针对算法中的 S 盒进行. ANT 算法采用 AND 操作作为唯一的非线性操作, 同传统的 S 盒算法相比, 其针对侧信道攻击的防护更容易实现且代价更小.

1.3 本文框架

本文章节内容安排如下: 第 2 节给出 ANT 的算法描述, 第 3 节介绍 ANT 的算法设计原理, 第 4 节和第 5 节分别给出 ANT 算法的安全性分析和软硬件实现结果. 最后, 第 6 节总结全文.

2 ANT 算法描述

本节将给出 ANT 系列分组密码算法的详细说明. ANT 系列分组密码算法采用了经典的 Feistel 结构, 其所有分组可用 $ANT-2n/m$ 表示. 不同分组的相应参数在表 1 中具体给出.

表 1 ANT 系列分组密码算法相应参数
Table 1 Parameters for ANT family block cipher

分组比特长度 ($2n$)	密钥比特长度 (m)	字比特长度 (n)	轮数 (R)	算法名称
128	128	64	46	ANT-128/128
128	256	64	48	ANT-128/256
256	256	128	74	ANT-256/256

本文中采用的一些符号表示如下:

- $x = x_{n-1}||x_{n-2}||\cdots||x_0$: n 比特变量 x (从高位到低位).
- $x \ll a$: 变量 x 左移位 a 比特.
- $x \lll b$: 变量 x 左循环移位 b 比特.
- $x \oplus y$: 变量 x 和变量 y 进行异或操作.
- $x \odot y$: 变量 x 和变量 y 进行与操作.

2.1 轮函数

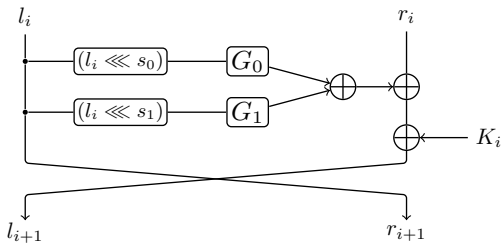


图 1 ANT 算法轮函数
Figure 1 Round function of ANT

ANT 算法的轮函数仅采用 AND, Rotation, XOR 三种操作, 结构如图 1 所示, 可以由如下公式表示:

$$(l_{i+1}, r_{i+1}) = R_{K_i}(l_i, r_i) = (G_0(l_i \lll s_0) \oplus G_1(l_i \lll s_1) \oplus r_i \oplus K_i, l_i)$$

其中, 轮数变量 i 有 $0 \leq i < R$. 注意为保证加解密一致, 最后一轮的左右支经过轮函数之后不进行交换. 循环移位参数 (s_0, s_1) 在 ANT 所有的分组中均设置为 $(3, 16)$. G_0 和 G_1 为比特级的非线性函数, 包含 2 层, 2 层之间是一层比特级的置换. 在具体介绍 G_0 和 G_1 之前, 先引入如下表示:

令 $x^0 = x_{n-1}^0 \| x_{n-2}^0 \| \cdots \| x_0^0$ 表示 G_0/G_1 的 n 比特输入变量, $y^0 = y_{n-1}^0 \| y_{n-2}^0 \| \cdots \| y_0^0$ 表示 G_0/G_1 第一层的 n 比特输出变量. 类似的, $x^1 = x_{n-1}^1 \| x_{n-2}^1 \| \cdots \| x_0^1$ 和 $y^1 = y_{n-1}^1 \| y_{n-2}^1 \| \cdots \| y_0^1$ 分别表示 G_0/G_1 第二层的 n 比特输入变量和输出变量. PERM 表示 G_0/G_1 中所采用的相同的比特级置换. 如下给出非线性函数 G_0 和 G_1 的公式表达 (其中下标变量 j 有 $0 \leq j < n$), 并在附录中给出 ANT-128 所采用的 G_0/G_1 的示意图 (见附录中的图 6 和图 7).

对于 G_0 :

$$y^1 = G_0(x^0)$$

先经过 G_0 中的第一层

$$y_j^0 = \begin{cases} (x_{j+3}^0 \odot x_{j+2}^0) \oplus x_j^0, & \text{if } j \bmod 4 \equiv 0, \\ x_j^0, & \text{otherwise} \end{cases}$$

再经过中间的置换 PERM

$$x_{\text{PERM}(j)}^1 = y_j^0$$

最后经过第二层

$$y_j^1 = \begin{cases} (x_{j+3}^1 \odot x_{j+2}^1) \oplus x_j^1, & \text{if } j \bmod 4 \equiv 0, \\ x_j^1, & \text{otherwise} \end{cases}$$

类似的, 对于 G_1 :

$$y^1 = G_1(x^0)$$

先经过 G_1 中的第一层

$$y_j^0 = \begin{cases} (x_{j+2}^0 \odot x_{j+1}^0) \oplus x_j^0, & \text{if } j \bmod 4 \equiv 1, \\ x_j^0, & \text{otherwise} \end{cases}$$

再经过中间的置换 PERM

$$x_{\text{PERM}(j)}^1 = y_j^0$$

最后经过第二层

$$y_j^1 = \begin{cases} (x_{j+2}^1 \odot x_{j+1}^1) \oplus x_j^1, & \text{if } j \bmod 4 \equiv 1, \\ x_j^1, & \text{otherwise} \end{cases}$$

对于不同 ANT 分组中所采用的置换 PERM, 其表达式如下 (PERM 置换的表格形式见表 2 和表 3),

对于 ANT-128:

$$\text{PERM}(j) = \begin{cases} (j + 58) \bmod 64, & \text{if } j \bmod 4 \equiv 3, \\ (j + 54) \bmod 64, & \text{if } j \bmod 4 \equiv 2, \\ (j + 30) \bmod 64, & \text{if } j \bmod 4 \equiv 1, \\ (j + 2) \bmod 64, & \text{if } j \bmod 4 \equiv 0 \end{cases}$$

表 2 ANT-128 的 PERM
Table 2 PERM in ANT-128

j	PERM(j)	j	PERM(j)	j	PERM(j)	j	PERM(j)
0	2	16	18	32	34	48	50
1	31	17	47	33	63	49	15
2	56	18	8	34	24	50	40
3	61	19	13	35	29	51	45
4	6	20	22	36	38	52	54
5	35	21	51	37	3	53	19
6	60	22	12	38	28	54	44
7	1	23	17	39	33	55	49
8	10	24	26	40	42	56	58
9	39	25	55	41	7	57	23
10	0	26	16	42	32	58	48
11	5	27	21	43	37	59	53
12	14	28	30	44	46	60	62
13	43	29	59	45	11	61	27
14	4	30	20	46	36	62	52
15	9	31	25	47	41	63	57

表 3 ANT-256 的 PERM
Table 3 PERM in ANT-256

j	PERM(j)	j	PERM(j)	j	PERM(j)	j	PERM(j)
0	2	32	34	64	66	96	98
1	63	33	95	65	127	97	31
2	120	34	24	66	56	98	88
3	125	35	29	67	61	99	93
4	6	36	38	68	70	100	102
5	67	37	99	69	3	101	35
6	124	38	28	70	60	102	92
7	1	39	33	71	65	103	97
8	10	40	42	72	74	104	106
9	71	41	103	73	7	105	39
10	0	42	32	74	64	106	96
11	5	43	37	75	69	107	101
12	14	44	46	76	78	108	110
13	75	45	107	77	11	109	43
14	4	46	36	78	68	110	100
15	9	47	41	79	73	111	105
16	18	48	50	80	82	112	114
17	79	49	111	81	15	113	47
18	8	50	40	82	72	114	104
19	13	51	45	83	77	115	109
20	22	52	54	84	86	116	118
21	83	53	115	85	19	117	51
22	12	54	44	86	76	118	108
23	17	55	49	87	81	119	113
24	26	56	58	88	90	120	122
25	87	57	119	89	23	121	55
26	16	58	48	90	80	122	112
27	21	59	53	91	85	123	117
28	30	60	62	92	94	124	126
29	91	61	123	93	27	125	59
30	20	62	52	94	84	126	116
31	25	63	57	95	89	127	121

对于 ANT-256:

$$\text{PERM}(j) = \begin{cases} (j + 122) \bmod 128, & \text{if } j \bmod 4 \equiv 3, \\ (j + 118) \bmod 128, & \text{if } j \bmod 4 \equiv 2, \\ (j + 62) \bmod 128, & \text{if } j \bmod 4 \equiv 1, \\ (j + 2) \bmod 128, & \text{if } j \bmod 4 \equiv 0 \end{cases}$$

2.2 密钥生成算法

ANT 分组密码算法可采用 $2n$ 或者 $4n$ 比特长度的主密钥 K , 因此使用如下 2 种基于线性反馈移位寄存器 (LFSR) 的密钥扩展方案.

对于 **ANT- $2n/2n$** : $2n$ 比特的的主密钥 $K = k_{2n-1} \| k_{2n-2} \| \cdots \| k_0$ 可以分成 2 个字 (高 n 比特和低 n 比特),

$$K_1 = k_{2n-1} \| k_{2n-2} \| \cdots \| k_n, K_0 = k_{n-1} \| k_{n-2} \| \cdots \| k_0$$

K_1, K_0 作为图 2 中 LFSR 的初始状态. 每次根据第 i 轮子密钥 K_i 和第 $(i + 1)$ 轮子密钥 K_{i+1} 可以更新出第 $(i + 2)$ 轮子密钥 K_{i+2} , 其中 $0 \leq i < R - 2$. LFSR 在进行更新时, $(i + 1)$ 作为轮常数, 当前寄存器 K_{i+1} 中的状态经过 Feistel 结构的 A 操作 (见图 3) 迭代更新 3 次. 注意不同分组下, A 操作的输入分成了 8 个 $\frac{n}{8}$ 比特的变量, $X_7 \| \cdots \| X_0$, 其中的移位参数 (t_0, t_1) 在表 4 中给出. 如下给出 ANT- $2n/2n$ 密钥生成算法的更新表达式:

$$K_{i+2} = A^3(K_{i+1}) \oplus K_i \oplus (i + 1)$$

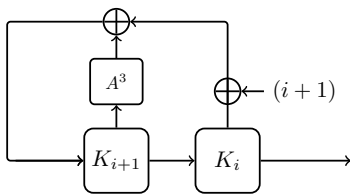


图 2 ANT- $2n/2n$ 密钥生成算法
Figure 2 Key schedule of ANT- $2n/2n$

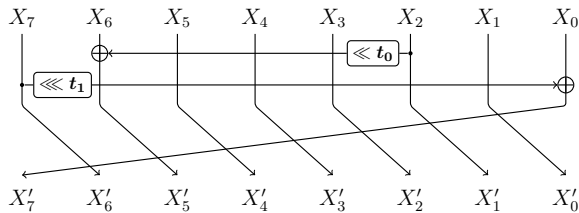


图 3 A 操作
Figure 3 Operation A

表 4 A 操作中的 (t_0, t_1) 移位参数
Table 4 Parameters (t_0, t_1) in A operation

t_0	t_1	算法名称
7	1	ANT-128/128
7	1	ANT-128/256
15	1	ANT-256/256

对于 **ANT- $2n/4n$** : 类似的, $4n$ 比特的的主密钥 $K = k_{4n-1} \| k_{4n-2} \| \cdots \| k_0$ 可以分成 4 个字,

$$K_3 = k_{4n-1} \| k_{4n-2} \| \cdots \| k_{3n}, \quad K_2 = k_{3n-1} \| k_{3n-2} \| \cdots \| k_{2n}, \\ K_1 = k_{2n-1} \| k_{2n-2} \| \cdots \| k_n, \quad K_0 = k_{n-1} \| k_{n-2} \| \cdots \| k_0$$

K_3, K_2, K_1, K_0 作为图 4 中 LFSR 的初始状态. 每次根据第 i 轮子密钥 K_i , 第 $(i + 1)$ 轮子密钥 K_{i+1} , 第 $(i + 2)$ 轮子密钥 K_{i+2} 和第 $(i + 3)$ 轮子密钥 K_{i+3} 可以更新出第 $(i + 4)$ 轮子密钥 K_{i+4} , 其中 $0 \leq i < R - 4$. LFSR 在进行更新时, $(i + 1)$ 作为轮常数, 当前寄存器 K_{i+3} 中的状态经过 Feistel 结构的 A 操作 (见图 3) 迭代更新 3 次. 注意不同分组下, A 操作的输入分成了 8 个 $\frac{n}{8}$ 比特的变量, $X_7 \| \cdots \| X_0$,

移位参数同样见表 4. 如下给出 ANT-2n/4n 密钥生成算法的更新表达式:

$$K_{i+4} = A^3(K_{i+3}) \oplus K_{i+1} \oplus K_i \oplus (i+1)$$

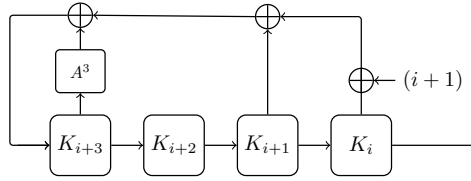


图 4 ANT-2n/4n 密钥生成算法
Figure 4 Key schedule of ANT-2n/4n

3 ANT 算法设计原理

3.1 Feistel 结构结合 Expand-then-compress 思想

Feistel 结构是一种经典的分组密码结构. 与 SPN 结构相比, 具有加解密一致的优势, 有利于减小硬件实现代价. 但由于 Feistel 结构每次只更新部分数据, 导致基于 Feistel 结构的算法同基于 SPN 结构的算法相比扩散速度较慢. 因此, ANT 算法在设计的过程中, 采用了 Expand-then-compress 的设计思想, 将算法的一支先进行扩展复制, 再分别进入两个非线性函数 G_0 和 G_1 . 一方面, 可以增加扩散速度; 另一方面, 可以并行处理以减小轮函数时延. 这种设计思想被许多分组密码所采用, 如 DES^[13]、PICARO^[14] 和 SIMON^[15] 等.

3.2 比特级非线性函数 G_0/G_1

非线性函数 G_0 和 G_1 均为 2 层结构, 并且均采用比特级的设计, 仅包含 AND, Rotation 和 XOR 三种操作. 相比于传统的 S 盒算法, 这样可以极大地减小硬件面积, 同时针对侧信道防护的代价很小. 除去中间一层所采用的相同置换 PERM, G_0 和 G_1 分别由相近的 nibble 基本单元组成, 如图 5.

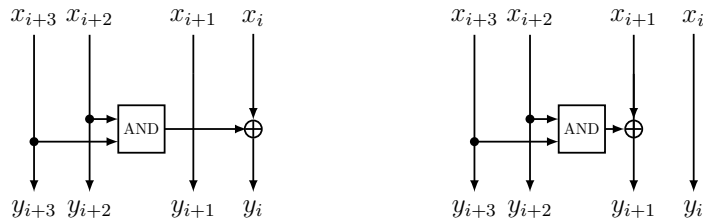


图 5 G_0 (左) 和 G_1 (右) 中的 nibble 基本单元
Figure 5 Basic nibbles in G_0 (left) and G_1 (right)

由于 G_0 和 G_1 中的 nibble 基本单元非常简单, 仅有 1 个非线性操作 AND 和 1 个线性操作 XOR 组成. 因此, 置换 PERM 的选择直接影响到算法的安全性, 其选择的原则有以下几点:

- 保证每个比特经过 G_0/G_1 后都经过非线性操作.
- 保证经过压缩操作之后每个比特的代数度至少为 2 次.
- 结合循环移位参数 (s_0, s_1) ¹ 达到较好的扩散效果.
- 方便进行 bitslice 软件实现.

¹循环移位参数 (s_0, s_1) 的选取主要是避免相关性对安全性评估造成影响, 所以 s_0, s_1 的差值非 4 的倍数. 通过穷搜在小分组的特定轮数下所有循环移位参数, 以对差分/线性抵抗能力为指标进行选取, 最终确定为 (3,16), 并将此参数应用到大分组上, 发现仍能达到较好的安全性.

从结果看, ANT 算法的各个分组均达到了较好的扩散性. 其中, ANT-128 的全扩散轮数为 5 轮, ANT-256 的全扩散轮数为 7 轮. 考虑到 ANT 算法的轻量级设计和 Feistel 结构, 这个扩散速度已经较快 (SIMON-128 算法的全扩散轮数为 13 轮).

3.3 密钥生成算法

ANT 算法的密钥生成算法在设计时主要考虑以下几点: (1) 硬件代价小; (2) 扩散性好; (3) 匹配轮函数的时延. 因此, 我们采用了近年来逐渐被广泛采用的线性的密钥生成算法, 通过 LFSR 的形式来更新子密钥. 结合 Feistel 结构的 A 操作, 通过迭代多次来换取较大硬件面积才能够达到的扩散效果, 这种方式最早运用在基于 SPN 结构的 CUBE 分组密码算法^[16]中. 我们针对 ANT 算法进行扩展, 在保证较好扩散效果的同时, 进一步减小硬件代价 (比如令变量经过 t_0 移位操作后只剩下 1 比特).

4 ANT 算法安全性分析

4.1 差分分析和线性分析

差分分析 (DC)^[17] 和线性分析 (LC)^[18] 均是经典的密码分析方法. 为了展示 ANT 系列分组密码算法对差分攻击和线性攻击的抵抗能力, 在表 5 中给出了不同分组最优的差分特征和线性特征的搜索情况 (利用 SAT/SMT 技术进行搜索). 由此可以得到, ANT-128 不存在超过 27 轮的有效差分或线性特征 (28 轮的差分特征的下界为 $49 + 42 + 42 = 133$), ANT-256 不存在超过 47 轮的有效差分或线性特征 ($45 \times 6 = 270$).

表 5 ANT 算法差分特征和线性特征搜索结果 (概率以 $-\log_2 p$ 形式给出, 线性特征使用相关度)
Table 5 Searching results of differential/linear characteristics for ANT (probability is given as $-\log_2 p$ and the correlation is used for linear trail)

特征	分组	轮数								
		2	3	4	5	6	7	8	9	10
差分	ANT-128	2	4	8	12	20	28	36	42	49
	ANT-256	2	4	8	12	20	28	45	-	-
线性	ANT-128	1	2	4	6	10	14	20	26	≥ 28
	ANT-256	1	2	4	6	10	14	23	-	-

我们还对 ANT 算法的 differential 和 linear hull 效应进行了测试, 结果发现 ANT 系列算法的 differential 和 linear hull 效应不显著. 根据搜索得到 ANT-128 的 6 轮最优差分路线 (概率为 2^{-20}) 和 6 轮最优线性路线 (相关性为 2^{-10}). 首先, 利用 SAT/SMT 技术进行路线堆积^[19], 发现理论堆积的效果不显著. 然后, 随机取 100 个密钥, 在每个密钥下选取 2^{24} 的数据量对路线进行验证. 实验结果与理论堆积的结果相符合. 因此, ANT 系列分组密码算法有较好的差分和线性抵抗能力, 设置的轮数足够抵抗差分攻击和线性攻击.

4.2 不可能差分分析和零相关线性分析

不可能差分分析 (IDC)^[20, 21] 和零相关线性分析 (ZC)^[22] 分别是对差分分析和线性分析方法的拓展. 同样利用 SAT/SMT 技术, 我们利用比特级的搜索方式^[23] 对不同分组 ANT 的不可能差分 and 零相关线性路线进行搜索, 结果如表 6.

表 6 ANT 算法搜索到的最长 IDC 和 ZC 区分器轮数
Table 6 Longest IDC and ZC distinguishers found for ANT

IDC 路线轮数	ZC 路线轮数	算法名称
9	10	ANT-128
13	13	ANT-256

因此, 不可能差分和零相关线性攻击对 ANT 系列分组密码算法不够成威胁. 如下给出一条 ANT-128 的 9 轮 IDC 路线示例:

$(l_0, r_0) = (0000000000000000, 8000000000000000) \not\Rightarrow (l_9, r_9) = (4000000000000000, 0000000000000000)$

4.3 积分分析

积分攻击^[24]利用特定的明文集合在密文处的非随机的代数性质来区分随机置换和目标密码算法. 本文利用 SAT/SMT 技术, 基于比特级的可分性^[25]对不同分组的 ANT 算法进行积分路线搜索. 对于 ANT-128, 经搜索发现不存在超过 16 轮的积分路线; 对于 ANT-256, 经过搜索发现不存在超过 21 轮的积分路线. 因此, ANT 系列分组密码算法的轮数足以抵抗积分攻击.

5 ANT 算法性能分析

5.1 软件性能

我们对 ANT 系列分组密码算法进行了 64-bit Windows 下的 CBC 单路加/解密速度测试, CBC 多路解密速度测试和 32-bit ARM 下的 CBC 单路加/解密速度测试. 测试过程采用 256 字节短数据进行一次 CBC 模式下的加密, 包含了密钥生成算法的时间. 如此进行多次并且每次加密均运行一次完整的密钥生成算法, 最后取速率的平均值. 测试环境分别为:

- Windows7, i7-6700, 8 GB DDR4 2400M. VS 2017, x64, avx2.
- STM32F103ZET6, 主频 72 MHz. EWARM8.2.

具体测试结果见表 7. 可以发现, 针对 bitslice 而设计的比特级轮函数结构, 使 ANT 算法具有较好的多路运行速率.

表 7 ANT 算法软件速率优化 C 实现
Table 7 Software performance of ANT under C optimized implementations

实现平台	分组	CBC 单路加密/解密速率 (Mbps)	CBC 多路解密速率 (Mbps)
64-bit Windows	ANT-128/128	597	6065
	ANT-128/256	581	6137
	ANT-256/256	587	2635
32-bit ARM	ANT-128/128	2.2	-
	ANT-128/256	2.1	-
	ANT-256/256	2.2	-

5.2 硬件性能

我们利用 Verilog HDL 语言对 ANT 系列分组密码算法进行了基于单轮的实现, 采用 32 个分组数据进行加/解密, 且至少调用 1 次密钥扩展算法. 硬件仿真数据如表 8.

表 8 基于单轮实现的硬件指标 (综合采用 HJTC 110 nm 标准元件库)
Table 8 Hardware performance for round-based implementations (synthesized with HJTC 110 nm standard cell library)

对比项	ANT-128/128	ANT-128/256	ANT-256/256
运行周期 (Cycles)	2991	3121	4811
时钟约束 (ns)	2.14	2.07	2.14
加密速率 (Mbps)	1300.28	1288.24	1616.57
解密速率 (Mbps)	1260.05	1248.42	1566.94
面积 (um ²)	31 234.66	49 160.10	63 201.84
等效门电路数 (GE)	3220.40	5068.57	6516.36
加密吞面比 (Mbps/um ²)	0.0416	0.0262	0.0256
解密吞面比 (Mbps/um ²)	0.0403	0.0254	0.0248

可以观察到, 得益于简单的 AND-RX 结构, ANT 算法的硬件实现代价非常小, 再结合上 Feistel 结

构加解密一致的特性,使得 ANT 算法具有优异的硬件实现性能,各版本均具有较高的吞面比. ANT-128/128 分组加解密基于轮实现仅需硬件面积 3220 GE.

6 总结

本文给出了一款新的分组密码算法—ANT,其设计目标是:相比于国际上传统的分组密码算法,更适合轻量级实现,具有优异吞面比,便于侧信道防护且代价小,同时适合 bitslice 软件多路实现的分组密码算法.因此在设计过程中,ANT 算法采用了经典的 Feistel 结构并结合 Expand-then-compress 的设计理念.在关键的轮函数上,采用了比特级的设计(与操作,异或操作,置换操作和循环移位操作),与传统利用 S 盒作为非线性组件的算法相比,其硬件面积小且侧信道防护代价小.一些组件如比特级置换的选取也考虑了软件多路 bitslice 的实现性能.密钥生成算法的设计,一方面采用了近年来应用广泛的线性的密钥生成算法,达到了较好的扩散性.另一方面,硬件实现代价小并且匹配了轮函数的时延.针对现有常见攻击方法的安全性分析也表明 ANT 系列分组密码算法具有足够的安全冗余.

References

- [1] DAEMEN J, RIJMEN V. AES proposal: Rijndael[OL]. 1999. https://cs.ru.nl/~joan/papers/JDA_VRI_Rijndael_V2_1999.pdf
- [2] GUO J, PEYRIN T, POSCHMANN A, et al. The LED block cipher[C]. In: Cryptographic Hardware and Embedded Systems—CHES 2011. Springer Berlin Heidelberg, 2011: 326–341. [DOI: 10.1007/978-3-642-23951-9_22]
- [3] WU W L, ZHANG L. LBlock: A lightweight block cipher[C]. In: Applied Cryptography and Network Security—ACNS 2011. Springer Berlin Heidelberg, 2011: 327–344. [DOI: 10.1007/978-3-642-21554-4_19]
- [4] ZHANG W, BAO Z, LIN D, et al. RECTANGLE: A bit-slice lightweight block cipher suitable for multiple platforms[J]. Science China Information Sciences, 2015, 58(12): 1–15. [DOI: 10.1007/s11432-015-5459-7]
- [5] BORGHOFF J, CANTEAUT A, GÜNEYSU T, et al. PRINCE: A low-latency block cipher for pervasive computing applications[C]. In: Advances in Cryptology—ASIACRYPT 2012. Springer Berlin Heidelberg, 2012: 208–225. [DOI: 10.1007/978-3-642-34961-4_14]
- [6] GÉRARD B, GROSSO V, NAYA-PLASENCIA M, et al. Block ciphers that are easier to mask: How far can we go?[C]. In: Cryptographic Hardware and Embedded Systems—CHES 2013. Springer Berlin Heidelberg, 2013: 383–399. [DOI: 10.1007/978-3-642-40349-1_22]
- [7] BOGDANOV A, KNUDSEN L R, LEANDER G, et al. PRESENT: An ultra-lightweight block cipher[C]. In: Cryptographic Hardware and Embedded Systems—CHES 2007. Springer Berlin Heidelberg, 2007: 450–466. [DOI: 10.1007/978-3-540-74735-2_31]
- [8] SHIRAI T, SHIBUTANI K, AKISHITA T, et al. The 128-bit blockcipher CLEFIA (Extended Abstract)[C]. In: Fast Software Encryption—FSE 2007. Springer Berlin Heidelberg, 2007: 181–195. [DOI: 10.1007/978-3-540-74619-5_12]
- [9] NIST. NIST Lightweight Standardization Process[OL]. <https://csrc.nist.gov/Projects/lightweight-cryptography>
- [10] BEIERLE C, JEAN J, KÖLBL S, et al. The SKINNY family of block ciphers and its low-latency variant MANTIS[C]. In: Advances in Cryptology—CRYPTO 2016, Part II. Springer Berlin Heidelberg, 2016: 123–153. [DOI: 10.1007/978-3-662-53008-5_5]
- [11] BANIK S, PANDEY S K, PEYRIN T, et al. GIFT: A small present[C]. In: Cryptographic Hardware and Embedded Systems—CHES 2017. Springer Cham, 2017: 321–345. [DOI: 10.1007/978-3-319-66787-4_16]
- [12] SONG L, HUANG Z, and YANG Q. Automatic differential analysis of ARX block ciphers with application to SPECK and LEA[C]. In: Information Security and Privacy—ACISP 2016, Part II. Springer Cham, 2016: 379–394. [DOI: 10.1007/978-3-319-40367-0_24]
- [13] Data encryption standard (DES)[S/OL]. Federal Information Processing Standards Publication 46-3. 1999. <https://csrc.nist.gov/csrc/media/publications/fips/46/3/archive/1999-10-25/documents/fips46-3.pdf>
- [14] PIRET G, ROCHE T, CARLET C. PICARO—A block cipher allowing efficient higher-order side-channel resistance[C]. In: Applied Cryptography and Network Security—ACNS 2012. Springer Berlin Heidelberg, 2012: 311–328. [DOI: 10.1007/978-3-642-31284-7_19]
- [15] BEAULIEU R, SHOR D, SMITH J, et al. The SIMON and SPECK lightweight block ciphers[C]. In: Proceedings of the 52nd Annual Design Automation Conference. ACM, 2015: Article No. 175. [DOI: 10.1145/2744769.2747946]

- [16] BERGER T P, FRANCO J, MINIER M. Cube cipher: A family of quasi-involutive block ciphers easy to mask[C]. In: Codes, Cryptology, and Information Security—C2SI 2015. Springer Cham, 2015: 89–105. [DOI: 10.1007/978-3-319-18681-8_8]
- [17] BIHAM E, SHAMIR A. Differential cryptanalysis of DES-like cryptosystems[C]. In: Advances in Cryptology—CRYPTO '90. Springer Berlin Heidelberg, 1990: 2–21. [DOI: 10.1007/3-540-38424-3_1]
- [18] MATSUI M. Linear cryptanalysis method for DES cipher[C]. In: Advances in Cryptology—EUROCRYPT '93. Springer Berlin Heidelberg, 1993: 386–397. [DOI: 10.1007/3-540-48285-7_33]
- [19] SOOS M, NOHL K, CASTELLUCCIA C. Extending SAT solvers to cryptographic problems[C]. In: Theory and Applications of Satisfiability Testing—SAT 2009. Springer Berlin Heidelberg, 2009: 244–257. [DOI: 10.1007/978-3-642-02777-2_24]
- [20] BIHAM E, BIRYUKOV A, SHAMIR A. Cryptanalysis of Skipjack reduced to 31 rounds using impossible differentials[C]. In: Advances in Cryptology—EUROCRYPT '99. Springer Berlin Heidelberg, 1999: 12–23. [DOI: 10.1007/3-540-48910-X_2]
- [21] KNUDSEN L R. Deal: A 128-bit block cipher[R]. Technical Report 151, Department of Informatics, University of Bergen, Bergen, Norway, 1998.
- [22] BOGDANOV A, RIJMEN V. Linear hulls with correlation zero and linear cryptanalysis of block ciphers[J]. Design. Codes and Cryptography, 2014, 70(3): 369–383. [DOI: 10.1007/s10623-012-9697-z]
- [23] CUI T T, CHEN S Y, FU K, et al. New automatic search tool for impossible differentials and zero-correlation linear approximations[J]. SCIENCE CHINA Information Sciences. To be published. [DOI: 10.1007/s11432-018-1506-4]
- [24] KNUDSEN L R, WAGNER D A. Integral cryptanalysis[C]. In: Fast Software Encryption—FSE 2002. Springer Berlin Heidelberg, 2002: 112–127. [DOI: 10.1007/3-540-45661-9_9]
- [25] XIANG Z J, ZHANG W T, BAO Z Z, et al. Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers[C]. In: Advances in Cryptology—ASIACRYPT 2016, Part I. Springer Berlin Heidelberg, 2016: 648–678. [DOI: 10.1007/978-3-662-53887-6_24]

作者信息



陈师尧 (1992–), 湖北襄阳人, 博士生. 主要研究领域为对称密码算法的安全性分析.
sychen@mail.sdu.edu.cn



樊燕红 (1979–), 山东聊城人, 博士生. 主要研究领域为对称密码算法的安全性分析和实现.
fanyh@mail.sdu.edu.cn



付勇 (1981–), 山东寿光人, 副研究员. 主要研究领域为密码工程.
yongfu0976@163.com



黄鲁宁 (1979–), 山东海阳人, 助理研究员. 主要研究领域为对称密码算法的安全性分析.
lnhuang@sdu.edu.cn



王美琴 (1974–), 宁夏银川人, 教授. 主要研究领域为对称密码算法的设计和分析.
mqwang@sdu.edu.cn

附录

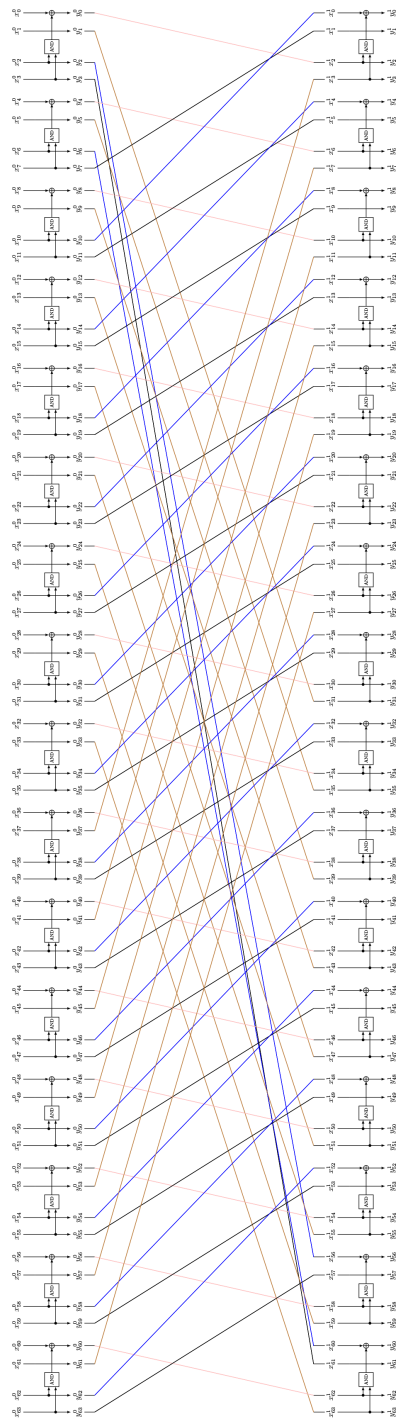


图 6 ANT-128 的 G_0
Figure 6 G_0 in ANT-128

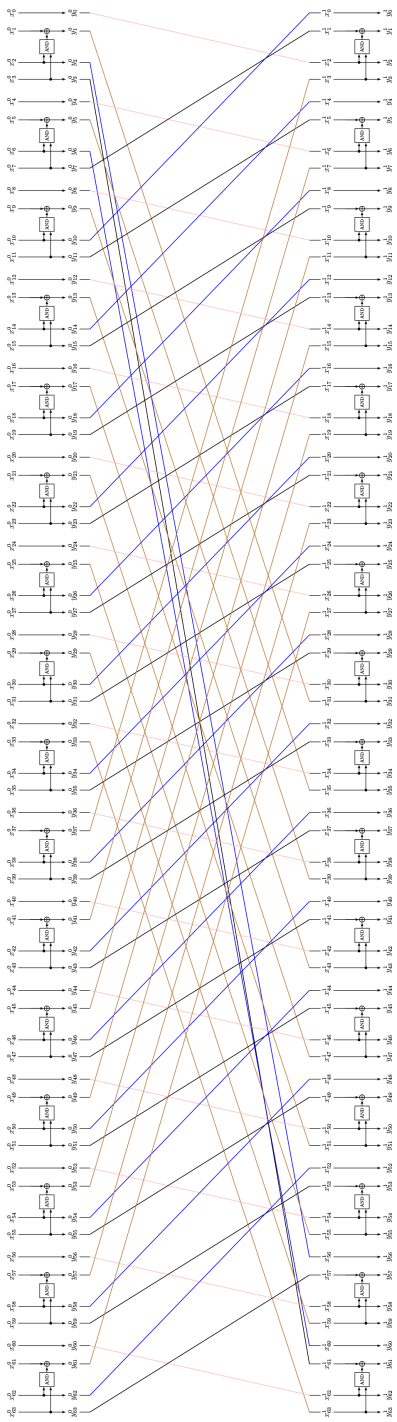


图 7 ANT-128 的 G_1
Figure 7 G_1 in ANT-128